

Citizen Complaint Portal

Government / Civic Tech | MERN Stack | Duration: 12 Hours

1. Overview

Build a web platform where citizens can report local civic problems — like broken roads, garbage piling up, streetlight failures, or water supply issues — and track how those complaints get resolved. Government staff (admins) can view these complaints, update their status, and manage them until they're fixed.

Think of it as a simple, transparent bridge between citizens and local authorities.

2. Problem Statement

In most areas, citizens have no easy way to report civic issues, and even when they do complain (phone calls, in-person visits), there's no way to track what happens next. This leads to unresolved problems and frustrated citizens.

Your task: build a portal that lets citizens report issues in under a minute, and lets officers manage and resolve them with full visibility for everyone.

3. Tech Stack

- Frontend: React.js, Next.js
- Backend: Express.js (Node.js)
- Database: MongoDB (Mongoose)
- Auth: JWT (JSON Web Tokens)
- Optional extras: Cloudinary/Multer for photo upload, Chart.js for stats, Socket.io for live status updates

4. User Roles (Keep it simple — 2 roles)

- Citizen — can sign up, submit complaints, view their own complaint status, upvote other complaints in their area.
- Officer/Admin — can view all complaints, filter by category/area, update status, add remarks.

5. Core Features Explained

Below, every feature is broken down into exactly what happens on screen, in that order — so you know what to build and in what sequence.

5.1 User Signup

- A new user (citizen) opens the Signup page and fills in: name, email, password, and role (citizen — officer accounts can be created separately or hardcoded for the demo).
- When they click 'Sign Up', the frontend sends this data to the backend.
- The backend checks the email isn't already used, encrypts (hashes) the password so it's never stored in plain text, and saves the new user in MongoDB.
- The backend sends back a success message, and the user is redirected to the Login page.

API endpoint(s) used: POST /api/auth/signup

5.2 User Login

- The user enters their email and password on the Login page and clicks 'Login'.
- The backend checks if the email exists and if the password matches (by comparing the hashed version).
- If correct, the backend creates a JWT token (like a digital ID card) and sends it back to the frontend.
- The frontend stores this token (in localStorage or context) and uses it to prove 'I'm logged in' on every future request.
- The user is redirected based on their role — citizens go to their Dashboard, officers go to the Officer Dashboard.

API endpoint(s) used: POST /api/auth/login

5.3 Submit a Complaint

- A logged-in citizen clicks 'Report a Complaint' and fills a form: title, category (dropdown: Road / Garbage / Water / Electricity / Other), description, and area or locality name.
- Optionally, they attach a photo of the issue (stretch goal — can skip if short on time).
- On clicking 'Submit', the frontend sends the form data plus the user's ID (taken from their login token) to the backend.
- The backend creates a new complaint entry in MongoDB with status automatically set to 'Pending', and links it to the citizen who filed it.
- The citizen sees a success message and is redirected to their complaint list, where the new complaint now appears.

API endpoint(s) used: POST /api/complaints

5.4 View My Complaints

- A citizen opens the 'My Complaints' page from their dashboard.
- The frontend asks the backend for all complaints where 'createdBy' matches the logged-in user's ID.
- The backend fetches only that user's complaints from MongoDB and sends them back.
- The frontend displays them as a list of cards, each showing title, category, status (Pending/In Progress/Resolved), and date filed.

API endpoint(s) used: GET /api/complaints/mine

5.5 Public Complaint Feed

- Any visitor (logged in or not) opens the 'Browse Complaints' page.
- The frontend requests all complaints from the backend (or filtered by area if a filter is applied).
- The backend returns complaints sorted by newest first (or by most upvotes, if you want a 'trending issues' feel).
- Each complaint card shows title, category, area, current status, and an upvote count/button.
- This helps citizens see if an issue is already reported before filing a duplicate one.

API endpoint(s) used: GET /api/complaints

5.6 Upvote a Complaint

- While browsing the public feed, a citizen sees a complaint that matches an issue they've also noticed.
- Instead of filing a new complaint, they click the 'Upvote' button on that card.
- The frontend sends a request to increase that complaint's upvote count by 1.
- The backend updates the 'upvotes' field in MongoDB for that specific complaint.
- The updated count reflects immediately on screen, so higher-upvoted complaints can be treated as higher priority.

API endpoint(s) used: PATCH /api/complaints/:id/upvote

5.7 Officer Dashboard (View All Complaints)

- An officer logs in and lands on the Officer Dashboard, which shows every complaint in the system as a table (not just their own).
- The frontend requests all complaints from the backend, along with filter options (category, area, status).
- The officer can type in a search box or select dropdown filters — the table updates to show only matching complaints.

- Clicking on any row opens the full complaint details (description, photo if any, citizen info, current status).

API endpoint(s) used: GET /api/complaints (with query filters like ?status=pending&category=road)

5.8 Update Complaint Status

- From the complaint detail view, the officer sees a status dropdown showing: Pending, In Progress, Resolved.
- The officer selects a new status and optionally types a short remark (e.g., 'Team assigned, will fix by Friday').
- On clicking 'Update', the frontend sends the new status and remark to the backend for that specific complaint ID.
- The backend updates that complaint's document in MongoDB with the new status and remark, plus a timestamp.
- The next time the citizen who filed it opens 'My Complaints', they see the updated status and remark automatically.

API endpoint(s) used: PATCH /api/complaints/:id/status

5.9 Search & Filter

- On the browse/dashboard pages, the user types a keyword into a search box, or picks values from category/status/area dropdowns.
- As filters change, the frontend either re-fetches from the backend with those filters as query parameters, or filters the already-loaded list on the frontend (fine for small datasets in a 12-hour build).
- The visible list of complaints updates to match — this makes it fast to find specific issues in a long list.

API endpoint(s) used: GET /api/complaints?search=...&category=...&status=...&area=...

5.10 Duplicate Complaint Detection

Before a citizen submits a new complaint, the system automatically checks if a similar complaint already exists in the same area and category — reducing spam and encouraging upvoting instead of re-filing.

- The citizen fills in the complaint form (category + area + description).
- Before they hit Submit, the frontend sends the category and area to the backend to check for existing complaints.
- The backend queries MongoDB for complaints with the same category and area that are still Pending or In Progress.
- If matches are found, a warning card appears: "A similar complaint already exists in your area — would you like to upvote it instead?"

- The citizen can either proceed to submit (if their issue is different) or click the matched complaint to upvote it directly.
- This keeps the complaint database clean and helps officers see genuinely high-priority issues.

Why it matters: Duplicate complaints dilute upvote counts and confuse officers. This feature nudges citizens toward consolidating issues, making the platform more accurate and useful.

API endpoint(s) used: GET /api/complaints?category=...&area=...&status=pending,in-progress

5.11 Complaint Priority Scoring

Instead of officers manually deciding which complaints to tackle first, the system auto-calculates a priority level based on upvote count and complaint age — and displays a colour-coded badge on every complaint card.

- When a complaint is fetched (on any listing page), the backend calculates a priority score using a simple formula:

Score = upvotes × 2 + daysSinceCreated

Score < 5 → Low

Score 5–15 → Medium

Score 16–30 → High

Score > 30 → Critical

- The priority badge is shown on every complaint card in the Officer Dashboard and Public Feed.
- Officers can sort or filter the dashboard by priority level to focus on the most urgent issues first.
- Priority is recalculated dynamically on every fetch — no need for a separate cron job.

Why it matters: Officers shouldn't have to manually judge urgency. A complaint ignored for 10 days with 20 upvotes should automatically surface to the top — this feature makes that happen without extra effort.

API endpoint(s) used: Computed server-side on GET /api/complaints (returned as priority field in each complaint object)

5.12 Citizen Feedback After Resolution

Once an officer marks a complaint as Resolved, the citizen who filed it is shown a prompt to rate how satisfied they are with the resolution. Officers can see aggregated satisfaction scores across all their resolved complaints.

- When a complaint's status is set to Resolved, a feedback flag (feedbackPending: true) is saved in the complaint document in MongoDB.

- The next time the citizen opens 'My Complaints', any resolved complaint with `feedbackPending: true` shows a prompt: "Was your issue resolved? Rate the response."
- The citizen selects a rating (Satisfied / Not Satisfied, or 1–5 stars) and optionally adds a short comment.
- The frontend submits this rating to the backend, which saves it on the complaint document and sets `feedbackPending: false`.
- On the Officer Dashboard, each officer can see an average satisfaction score across their resolved complaints — shown as a small stat card at the top of their dashboard.
- Complaints with a negative rating are flagged so admins can review if the resolution was genuinely adequate.

Why it matters: Without feedback, officers have no way of knowing if their resolution actually helped. This closes the loop and gives the system a real accountability layer — citizens feel heard, and officers know their work is being evaluated.

Tip: Store ratings as: `rating` (1–5 number), `feedbackComment` (string), `feedbackGiven` (boolean) on the Complaint model. No new collection needed.

API endpoint(s) used: PATCH /api/complaints/:id/feedback

5.13 AI Daily Briefing for Officers

When an officer opens their dashboard, an AI-generated summary card appears at the top — giving them an instant, plain-English overview of the current complaint situation without having to manually scan the entire table.

- When the Officer Dashboard loads, the frontend calls the backend to generate an AI summary.
- The backend fetches all complaints from MongoDB and computes key stats: total complaints, new today, overdue, resolved this week, top categories, hotspot areas.
- These stats are sent to the Claude API with a prompt asking it to write a natural, officer-friendly briefing in 3–5 sentences.
- The API returns a concise briefing — for example: "Today: 12 new complaints filed. 3 are critical — all Road issues in Sector G-9. 5 complaints are overdue by more than 3 days. Water supply complaints in Block D have surged this week with 18 upvotes combined. 7 complaints were resolved yesterday."
- This summary is displayed as a highlighted card at the top of the Officer Dashboard, refreshed each time the page loads.
- The officer gets instant situational awareness without reading through a full complaint list.

Why it matters: Officers are busy. A smart briefing that surfaces the most critical information in seconds makes the portal genuinely useful in a real government workflow — and demonstrates practical AI integration to judges.

Tip: Use any free AI API for this — Google Gemini API (gemini-1.5-flash, free tier), Groq API (llama3, free tier), or OpenRouter (free models available). Send a system prompt: "You are a concise government operations assistant. Summarize these complaint stats in 3–5 plain English sentences for an officer." Pass computed stats as a JSON string in the user message and display the text response in the dashboard card.

API endpoint(s) used: POST /api/ai/officer-summary (internal — calls Claude API, returns { summary: string })

5.14 Export Complaints to CSV

Officers can download the currently filtered complaint list as a .csv file — a must-have for real government reporting, audit trails, and sharing data with departments that don't have system access.

- On the Officer Dashboard, a "Download CSV" button sits alongside the existing filters.
- When clicked, the frontend sends the current active filters (category, status, area, search) to the backend export endpoint.
- The backend queries MongoDB with those same filters, then uses the json2csv library to convert the results into a properly formatted CSV file.
- The CSV includes columns: ID, Title, Category, Area, Status, Priority, Upvotes, Filed By (citizen name), Filed On, Last Updated, Officer Remark.
- The backend streams the file back with the correct Content-Disposition header so the browser triggers a download automatically.
- The downloaded file is named complaints_export_YYYY-MM-DD.csv for easy filing.

Why it matters: Government departments run on spreadsheets and reports. A civic portal that cannot export data will never be adopted in a real workflow. This feature takes about 1 hour to build and adds significant real-world credibility.

Tip: Install json2csv on the backend: npm install json2csv. Use the Parser class to convert an array of complaint objects to a CSV string, then set res.header("Content-Type", "text/csv") and res.attachment("complaints_export.csv") before sending.

API endpoint(s) used: GET /api/complaints/export?category=...&status=...&area=...

6. Frontend Page Routes (React Router)

This is the list of pages/screens your React app needs, and what the user sees and does on each one.

Route Path	Page Name	What the user sees / does here
/	Home / Landing Page	Short intro to the portal, buttons to Login or Sign Up, and maybe a preview of recent public complaints.
/signup	Signup Page	Form to create a new citizen account: name, email, password.
/login	Login Page	Form to log in with email and password. Redirects to the right dashboard based on role.
/dashboard	Citizen Dashboard	Landing page after citizen login. Shows quick links: Report a Complaint, My Complaints, Browse Complaints.
/complaints/new	Report a Complaint	Form to submit a new complaint (title, category, description, area, optional photo). Includes duplicate check warning.
/complaints/mine	My Complaints	List of complaints filed by the logged-in citizen, with current status and feedback prompt for resolved ones.
/complaints	Browse Complaints (Public Feed)	List of all complaints with search/filter, priority badges, and upvote buttons.
/complaints/:id	Complaint Detail Page	Full details of one complaint — description, photo, status, remarks, upvote count, priority badge.
/officer/dashboard	Officer Dashboard	AI briefing card at top. Table of all complaints with filters. Download CSV button.
/officer/complaints/:id	Officer Complaint Review	Same complaint details as above, plus controls for the officer to change status and add a remark.

7. Backend API Routes (Express)

This is the list of backend endpoints your Express server needs to build. Each one connects to a frontend action above.

Method + Route	Purpose (Plain English)	Who can use it
POST /api/auth/signup	Creates a new user account in the database.	Anyone (public)

Method + Route	Purpose (Plain English)	Who can use it
POST /api/auth/login	Checks email/password and returns a login token (JWT).	Anyone (public)
POST /api/complaints	Creates a new complaint, linked to the logged-in citizen.	Logged-in citizen
GET /api/complaints	Returns all complaints, optionally filtered. Includes computed priority field.	Anyone (public), officers see extra detail
GET /api/complaints/mine	Returns only the complaints filed by the currently logged-in citizen.	Logged-in citizen
GET /api/complaints/export	Returns filtered complaints as a downloadable .csv file.	Logged-in officer only
GET /api/complaints/:id	Returns full details of one specific complaint.	Anyone (public)
PATCH /api/complaints/:id/upvote	Increases the upvote count of a complaint by 1.	Logged-in citizen
PATCH /api/complaints/:id/status	Updates a complaint's status and remark.	Logged-in officer only
PATCH /api/complaints/:id/feedback	Saves a citizen's satisfaction rating after resolution.	Logged-in citizen (complaint owner)
POST /api/ai/officer-summary	Fetches complaint stats and returns an AI-generated briefing via Claude API.	Logged-in officer only

***Tip:** Use middleware to protect officer-only routes — check the JWT token and the user's role before allowing the status update to go through. This is a great, simple way to show you understand authentication and authorization (two different things!) to the judges.*

8. Stretch Goals (Only if core features are done early)

- Photo upload for complaints (proof of the issue) using Multer/Cloudinary.
- Map view showing complaint locations (can use a simple embedded map or just area-based pins).
- Admin analytics dashboard — number of complaints per category, resolved vs pending (use Chart.js).
- Email/SMS-style notification (can be simulated with an in-app notification) when status changes.

- Real-time status updates using Socket.io.

9. Suggested Database Structure (MongoDB Collections)

Users:

- name, email, password (hashed), role (citizen/officer)

Complaints:

- title, description, category, area, status (pending/in-progress/resolved), upvotes, imageUrl (optional), createdBy (user ID), officerRemark, createdAt
- priority — computed field (Low / Medium / High / Critical) based on upvotes and age
- feedbackRating (1–5), feedbackComment (string), feedbackGiven (boolean), feedbackPending (boolean)

10. 12-Hour Timeline

Time	Milestone
Hour 0 – 1	Team setup: repo, MongoDB Atlas cluster, Express server boilerplate, React app boilerplate. Finalize database schema including new fields (priority, feedbackRating, feedbackPending).
Hour 1 – 3	Backend: build User & Complaint models, auth routes (signup/login with JWT), complaint CRUD routes. Add priority score computation helper.
Hour 3 – 5	Frontend: build signup/login pages, complaint submission form with duplicate detection warning. Connect to backend APIs.
Hour 5 – 7	Frontend: public complaint feed with priority badges, My Complaints page with feedback prompt, upvote button, search/filter UI.
Hour 7 – 9	Officer dashboard: complaint table, AI briefing card (Claude API), status update controls, CSV export button, filters by category/area/status/priority.
Hour 9 – 10	Integration testing: test full flow (submit → duplicate check → view → officer updates → citizen feedback). Fix bugs.
Hour 10 – 11	UI polish: consistent styling, loading states, empty states, responsive layout, error handling.

Time	Milestone
Hour 11 – 12	Prepare demo: seed sample data, rehearse walkthrough, prepare 3–5 min presentation, deploy if possible.

11. Judging Criteria (Suggested)

Criteria	Weight	What we're looking for
Functionality	30%	Core features (submit, view, update status) + added features (duplicate check, priority, feedback, AI briefing, CSV export) work end-to-end without breaking.
Technical Execution	25%	Clean use of MERN stack, sensible database design, working auth. Claude API integration for AI briefing is a strong bonus.
UI/UX	20%	Clean, intuitive interface; easy for a citizen or officer to actually use. Priority badges and AI summary card add meaningful polish.
Real-world Relevance	15%	Does it genuinely solve the civic complaint problem? CSV export and satisfaction feedback significantly strengthen this score.
Presentation	10%	Clear demo, team explains their approach and choices well.

12. Rules & Notes

- AI tools (ChatGPT, Claude, Copilot, etc.) are allowed — they're a tool, not a shortcut around understanding your own code. Teams should be able to explain any part of their code to judges.
- Use MongoDB Atlas (cloud) so judges can view live data without local setup.
- Keep the UI simple and functional over flashy — working features beat unfinished polish.
- Deploy if time allows (Vercel/Netlify for frontend, Render/Railway for backend) — a live link is a strong bonus but not mandatory.
- Deployment is **mandatory**. The complete application must be deployed and accessible through a live URL before the hackathon submission.